SEARCHING (LECTURE)

FINDING SOMETHING IN A COLLECTION

- Is this username in our user database?
- Has anyone used this phrase in a previous work?
- Have I seen this DNA sequence pattern before?

LEARNING OBJECTIVES

- Use **linear search** to find an element inside a sequence
- Use **binary search** to find an element in a sorted sequence
- Know when to use linear vs. binary search.
- *Start thinking about how we can evaluate our programs in different ways than just "correct" and "incorrect"*

WHAT IS EFFICIENCY?

L11 Discuss with a partner and write down: what quality or qualities of a program make it more or less "efficient"?

OUR PRIMARY CONCERN TODAY: RUNTIME EFFICIENCY

All code takes time to run. A simple heuristic is that a function's runtime is proportional to the number of iterations of the loops it takes to execute.

```
def check_greater(x):
    if x > 5:
        print("it's bigger")
    else:
        print("it's smaller.")
```

Set number of instructions each time.

```
def check_all_greater(x):
    for val in x:
        if x > 5:
            print(f"{x} is bigger")
        else:
            print(f"{x} is smaller.")
```

Potentially infinite things happening (depending on `x`)!

Let's approximate "speed" with printed snakes: 🐍

Recall that `isupper()` is a method of strings that returns `True` only when all letters are uppercase. We could write our own implementation in a couple of ways...

```
def isupper_one(word):
  all_upper = True
  for letter in word:
    print("🐍")
    if not "A" <= letter <= "Z":
      all_upper = False
  return all_upper
```

```
def isupper_two(word):
  for letter in word:
    print("🐍")
    if not "A" <= letter <= "Z":
      return False
  return True
```

S7: How many snakes are printed if we run `isupper_one("SpRING")`?

S8: How many snakes are printed if we run `isupper_two("SpRING")`?

EARLY RETURNS

For problems that are just chained "and"/"or", you can often return early!

- `isupper()` -> "are all letters uppercase?" -> first letter is uppercase *and* second letter is uppercase *and* ... *and* last latter is uppercase
- logical `and` is `True` only when both arguments are `True`
- logical `and` is `False` if **any** of its arguments are `False`
- $\rightarrow$ safe to return `False` as soon as any boolean in the chain is `False`!

This is **boolean short-circuiting**, which is built into Python for safety and speed!

REVIEW: SEARCH AS A PROBLEM

Given a sequence and a target element, find a position at which that target is found inside of the sequence, or report that the target cannot be found (`-1`).

Both linear search and binary search will have the following signatures:

```
def xyz_search(seq, target):
    ...
```

QUICK ASIDE: THE RIGHT WAY

Several operations exist for lists/sequences:

- `.index(target)` performs a linear search to find the index of a `target` on a list.
  - Tragic: raises an error if `target` isn't found in the list 😟
- `in` performs a linear search to find whether a target element is contained in a list.

ACTIVITY: SEARCHING PRACTICE

What do the following function calls return?

- **(S9)** `linear_search([3, 1, 10, 17, 23, 31, -23], -23)`
- **(S10)** `linear_search([2, 2, 4, 8, 12, 14, 16, 32, 64], 2)`

THINKING CRITICALLY ABOUT LINEAR SEARCH

```
def linear_search(sequence, target):
  for idx, element in enumerate(sequence):
    print("🐍")
    if element == target:
      return idx
  return -1
```

L13 How many snakes get printed if...

- target is the first element in the sequence?
- target is the 10th element in the sequence?
- target is not in the sequence?

THINKING CRITICALLY ABOUT LINEAR SEARCH

```
def linear_search(sequence, target):
  for idx, element in enumerate(sequence):
    print("🐍")
    if element == target:
      return idx
  return -1
```

How many snakes get printed if...

- target is the first element in the sequence? **1**
- target is the 10th element in the sequence? **10**
- target is not in the sequence? **len(sequence)**

LINEAR SEARCH: PROPERTIES

Linear search is...

- **Complete:** we'll always get an answer
- **Correct:** we'll always get the right answer

These are desirable properties, but linear search is not always the most **efficient.**

- May require more time (~more iterations) than other searching algorithms for "average use"

FIVE THINGS TO THINK ABOUT

- Best case scenario
- Worst case scenario
- Average scenario
- completeness (always get an answer?)
- correctness (always get the right answer when you get an answer?)

C12 What are the best, worst, and average cases for linear search?

A CONTRASTING POINT OF VIEW

Here's a dumb searching algorithm called **Bogo Search**

```
from random import randrange
def bogo_search(sequence, target):
    while True:
        print("🤷")
        idx = randrange(len(sequence)) # picks a random index to look at
        if sequence[idx] == target:
            return idx
```

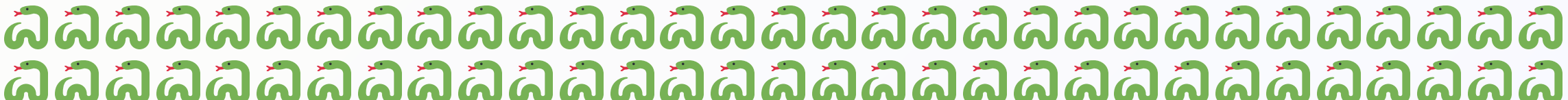
C14 What are the best, worst, and average cases for bogosearch? Is it complete and correct?

A CONTRASTING POINT OF VIEW

Here's a dumb searching algorithm called **Bogo Search**

```
from random import randrange
def bogo_search(sequence, target):
    while True:
        print("🐌")
        idx = randrange(len(sequence)) # picks a random index to look at
        if sequence[idx] == target:
            return idx
```

Not even **complete:** if we got unlucky, we could accidentally just look at the same (wrong) index infinitely many times in a row and never return.



BINARY SEARCH

BINARY SEARCH

Can we do better than linear search? Can we be...

- complete?
- correct?
- faster?

The answer is "yes, yes, and **sometimes."**

BINARY SEARCH

Used to search for a target value in an **(ascending) sorted sequence only**

- Compares the target with the value at the middle index (middle element)
  - If the middle element is the target element, then we're done!
  - If the target is less than the middle element, then we search for the target in the **left half of the sequence** (the positions before the middle element)
  - If the target is greater than the middle element, then we search the target in the **right half of the sequence** (the positions after the middle element)
- Repeat on the remaining search area of the sequence until
  - the element is found
  - there is no feasible search area left

BINARY SEARCH

Searching for `"Dustin"` in the sequence `names`:

CARYN	DEBBIE	DUSTIN	ELLIOT	JACQUIE	JON	RICH
0	1	2	3	4	5	6
low			middle			high

- `middle = (low + high) // 2 = 3`
- `names[middle]` is `"Elliot"`, which comes after `"Dustin"` alphabetically.
- So, if `"Dustin"` is present, it must be between positions `0` and `middle - 1`.
  - ➡️ shift `high` one to the left of `middle`

BINARY SEARCH

Searching for `"Dustin"` in the sequence `names`:

CARYN	DEBBIE	DUSTIN	ELLIOT	JACQUIE	JON	RICH
0	1	2	3	4	5	6
low	middle	high				

- `middle = (low + high) /// 2 = 1`
- `names[middle]` is `"Debbie"`, which comes before `"Dustin"` alphabetically.
- So, if `"Dustin"` is present, it must be between positions `middle + 1` and `2`.
  - ➡️ shift `low` one to the right of `middle`

BINARY SEARCH

Searching for `"Dustin"` in the sequence `names`:

CARYN	DEBBIE	DUSTIN	ELLIOT	JACQUIE	JON	RICH
0	1	2	3	4	5	6
		low, middle, high				

- `middle = (low + high) // 2 = 2`
- `names[middle]` is `"Dustin"`, which is the target element! So, we return `middle`.

BINARY SEARCH: SEARCHING FOR AN ELEMENT NOT PRESENT

Searching for `"Drew"` in the sequence `names`:

CARYN	DEBBIE	DUSTIN	ELLIOT	JACQUIE	JON	RICH
0	1	2	3	4	5	6
low			middle			high

- `middle = (low + high) // 2 = 3`
- `names[middle]` is `"Elliot"`, which comes after `"Drew"` alphabetically.
- So, if `"Drew"` is present, it must be between positions `0` and `middle - 1`.

BINARY SEARCH: SEARCHING FOR AN ELEMENT NOT PRESENT

Searching for `"Drew"` in the sequence `names`:

CARYN	DEBBIE	DUSTIN	ELLIOT	JACQUIE	JON	RICH
0	1	2	3	4	5	6
low	middle	high				

- `middle = (low + high) // 2 = 1`
- `names[middle]` is `"Debbie"`, which comes before `"Drew"` alphabetically.
- So, if `"Drew"` is present, it must be between positions `middle + 1` and `2`.

BINARY SEARCH: SEARCHING FOR AN ELEMENT NOT PRESENT

Searching for `"Drew"` in the sequence `names`:

CARYN	DEBBIE	DUSTIN	ELLIOT	JACQUIE	JON	RICH
0	1	2	3	4	5	6
		low, middle, high				

- `middle = (low + high) // 2 = 2`
- `names[middle]` is `"Dustin"`, which comes after `"Drew"` alphabetically.
- So, if `"Drew"` is present, it must be between positions `2` and `middle - 1`.

BINARY SEARCH: SEARCHING FOR AN ELEMENT NOT PRESENT

Searching for `"Drew"` in the sequence `names`:

CARYN	DEBBIE	DUSTIN	ELLIOT	JACQUIE	JON	RICH
0	1	2	3	4	5	6
	high	low				

- `high` is now less than `low`. The "feasible search area" is now totally empty.
- So, we return `-1` to indicate that the target was not found in the sequence.

BINARY SEARCH

```
def binary_search(sequence, target):
    low_index, high_index = 0, len(sequence) - 1
    while low_index <= high_index:
        middle_index = (low_index + high_index) // 2
        if target < sequence[middle_index]:      # middle too big:
            high_index = middle_index - 1        # discard right half
        elif target > sequence[middle_index]:    # middle too small:
            low_index = middle_index + 1         # discard left half
        else:
            return middle_index                  # found it! 😃
    return -1                                    # couldn't find it 😕
```

PROPERTIES OF BINARY SEARCH

- Binary Search is **complete** since each iteration of the `while` loop shrinks our feasible search area down to a point where we'll stop, or we return the index where we find the target.
- Binary Search is **correct** since we return the index of the target when we find it and we only return `-1` when the element could not have been present in the sequence.
  - This is only guaranteed if the sequence was sorted, though!
- Is Binary Search any more **efficient** than Linear Search?

COMPARING LINEAR & BINARY SEARCH

LINEAR SEARCH VS. BINARY SEARCH

- Binary search is faster "on average" than linear search 😄🎉🆒
  - Per iteration, binary search shrinks the feasible region by half the remaining elements, linear search only by one element.
  - In both cases, max number of iterations needed is bounded above by the number of iterations needed to shrink the feasible region to empty.
  - On average, binary search requires fewer iterations of the search loop
  - (when is binary search not faster then linear search?)

LINEAR SEARCH VS. BINARY SEARCH

Runtime analysis: how many iterations will it take to determine that the target is not in the sequence?

LENGTH OF THE SEQUENCE	LINEAR SEARCH	BINARY SEARCH
2	2	2
4	4	3
8	8	4
16	16	5
100	100	7

LINEAR SEARCH VS. BINARY SEARCH

Runtime analysis: how many iterations will it take to determine that the target is the first element of the sequence?

LENGTH OF THE SEQUENCE	LINEAR SEARCH	BINARY SEARCH
2	1	2
4	1	3
8	1	4
16	1	5
100	1	7

LINEAR SEARCH & BINARY SEARCH

Linear search is...

- Usable when your sequence is not sorted to start with
- As efficient as any search algorithm can be when you don't know anything about the sequence ahead of time

Binary search is...

- Only usable when your sequence is sorted to start with
- Significantly more efficient than linear search *on average.*